# 9ThirtyOne

Installation and Setup Guide

For AFROTC detachment administrators · Beta 1.3.0

---

### Before you start

Allow about 25 minutes for a first-time setup using Google Drive. You will need the detachment's Google account credentials, and nothing else — no Google Cloud project, no servers, no databases, no software licences.

Every procedure in this guide has been performed end to end against real Google Drive, on a Mac and an iPhone. Where something is awkward or unfinished, it is written down rather than smoothed over — see *What this version cannot do yet*.

## What this guide covers

9ThirtyOne is a web app that runs in a browser and stores every record in a Google Drive folder your detachment owns. You get the app from us, at one fixed address; your records never come to us. There is no server of ours and no shared database — which is why setup involves your own Google account rather than an account with us.

| | |
|---|---|
| 1 | **Decide where your data will live** — three options, and how to choose |
| 2 | **Prepare the Google account** — the account and the Drive folder; the Google registration is already done |
| 3 | **Open the app** — one address, nothing to install or host |
| 4 | **Run the setup wizard** — connect the app to your folder |
| 5 | **Claim the folder** — the first Google sign-in, and the access setting |
| 6 | **Deploy the submission server** — 15 minutes, and it changes what everything below costs |
| 7 | **Build the roster** — roles, and adding people by email or CSV |
| 8 | **Invite everyone** — one link, or a QR code on a projector |
| 9 | **Verify and hand over** — a checklist before first use |
| – | **Troubleshooting and reference** — common problems, folder layout, backups |

### What you will need

| ITEM | WHY |
|---|---|
| A new Google account, created for the detachment | Holds every record. Create a fresh one — Part 2a explains why a school or university account is the wrong choice here. |
| About 25 minutes, once | Most of it is the submission server in Part 6. The app itself takes a few minutes, and Part 2 no longer asks for a Cloud project at all. |
| The app's address | Part 3. The app is already published and maintained for you — there is nothing to download, deploy or keep running. |
| A current browser | Chrome, Edge, Safari or Firefox, on any device. |

**You are not signing up for anything**

9ThirtyOne has no accounts with a vendor, no subscription and no data-processing agreement to sign, because no data ever reaches anyone but you. Everything below sets up *your* Google account to hold *your* detachment's feedback.

That holds even though the app itself comes from us. What you load from our address is the program, not a service: it runs entirely in your browser and writes only to the Drive folder you connect in Part 4. Nobody who publishes the app can read that folder.

## 1 Decide where your data will live

Pick one. You can change later, but moving records between options is a manual export and import, so it is worth choosing correctly now.

| OPTION | BEST FOR | WHAT IT MEANS |
|---|---|---|
| **Google Drive** RECOMMENDED | Any detachment with more than one device | Records live in a Drive folder on your Google account. Works on phones, tablets and computers, and everyone sees the same data. Needs the detachment Google account created in Part 2 — there is no registration to do any more. |
| **Synced folder** | A single office computer | Points at the 9ThirtyOne folder inside Google Drive for Desktop. Google's own client handles syncing. Desktop Chrome or Edge only — no phones. |
| **This device only** | Trying the app out | Everything stays in one browser. Nothing syncs, nothing is shared, and clearing site data erases it. Fine for evaluation — export a backup and import it once you move to Drive. |

> **Why a browser cannot simply be pointed at a Drive folder path**
>
> Web pages have no access to the file system, and a phone has no synced Drive folder at all. The Google Drive option therefore connects through Google's own Drive service rather than a folder path. This is the reason Part 2 exists.

**If you are evaluating the app**, choose *This device only*, skip to Part 4, and come back to Part 2 when you are ready to go live. Nothing is lost — *Moving trial data into Drive*, at the start of Part 9, explains how to carry it across.

## 2 Prepare the Google account

Do this once, on the Google account that will own the data. About five minutes — this part used to take twenty, and most of it has gone away.

### 2a. Create the detachment's Google account

Create a **new, ordinary Google account** for this — the kind anyone can make at `accounts.google.com`, ending in `@gmail.com`. Name it after the unit, not after a person. Do not use anyone's existing personal account.

> **Why a fresh account, and not somebody's own**
>
> Exactly one account in the whole detachment needs Google Drive permission: this one. Everybody else — cadre, cadets, the commander — reaches their data through the submission server you deploy in Part 6, and never grants Drive access at all.
>
> That makes this account the detachment's records rather than a person's. On somebody's own account the folder leaves with them: every few years the next person has to export everything and rebuild it elsewhere, and the receipts and the audit trail do not survive the move. An account named for the unit is handed over instead — the data never moves, only the password does.
>
> A new account also has no institutional policy attached, so setup always completes. It costs nothing and takes about two minutes.

> **If your school will give the detachment its own account**
>
> A Google Workspace account — the kind a university issues, usually ending in `.edu` — is the better home for this, if you can get one in the detachment's name. It belongs to the institution rather than to a password in a shared folder, and it carries higher Google limits. The one that applies here is email: a Workspace account may send to fifteen hundred recipients a day where an ordinary account may send to a hundred, which decides whether a detachment can tell a whole flight about new feedback more than once in a day. Google publishes the current figures under *Apps Script quotas* in its developer documentation.
>
> Do not hold up the install waiting for one. Every school sets its own policy on who may be issued an account and what it may be used for, and not every cadre member has a Google-backed address to begin with. Setup is identical either way, so start on a Gmail account today and move later if the school offers one.

1. **Create the account at `accounts.google.com`.**
   Name it after the unit rather than a person — `det025.feedback@gmail.com`, or similar. A personal account takes the data with it when that person moves on.

2. **Record the password where the detachment keeps shared credentials.**
   This account owns every record. Losing it means losing the folder, and the recovery options on a brand-new account are thin.

3 **Point the recovery phone and address at something the unit keeps.**

A recovery number belonging to whoever set the account up is the commonest way a detachment loses access to its own records. It stops working the term they leave, and nobody finds out until the day they are locked out. Use a unit phone or a shared address, and check it still answers at each handover.

4 **Hand the account over deliberately, and change the password when you do.**

Write down who holds it. Google may lock an account it believes has been compromised, and several people signing in from several places is the pattern it watches for — so keep the number of people who use it small, and answer its prompts when it asks whether the sign-in was yours.

> **You do not create a folder yourself**
>
> Earlier versions of this guide asked you to create a folder in Drive by hand and paste its address into the app. **Do not do that.** The app creates its own folder during setup in Part 4, and a folder you make by hand cannot be used.
>
> The reason is the permission the app asks for. It requests access to *files it creates itself* and nothing else — it cannot see the rest of your Drive and has no way to ask. A folder made by hand was not created by the app, so Google does not show it to the app at all. Creating its own folder is precisely what lets the app avoid asking for access to your entire Google Drive.
>
> Nothing else changes. The folder appears in your Drive, belongs to you, and you can open, move or delete it like any other. You will need its address in Part 6, and the app shows it to you the moment it is created.

### 2b. Nothing — the app is already registered with Google

Earlier versions of this guide spent about twenty minutes here: a Google Cloud project, the Drive API, an OAuth consent screen, a list of every cadet as a *test user*, and an OAuth client ID to copy out. **None of that is needed any more.**

9ThirtyOne is now a verified Google application, published and registered once for every detachment that uses it. Your detachment supplies the Google account and the Drive folder; the registration is ours and is already done.

> **What this removes, in practice**
>
> No Cloud console. No consent screen to configure. **No test-user list**, so the hundred-account cap is gone and nobody has to be added before they can sign in. No Client ID to copy, paste or mistype — the app carries it.
>
> Cadets and cadre now meet an ordinary Google sign-in showing the 9ThirtyOne name, rather than the "*Google hasn't verified this app*" warning every user used to see every time.

**Who is asked for Drive permission, and who never is**

Only the account that owns the folder is ever asked, and only while you are setting it up in Part 4. The permission is `drive.file` — files this app created, and nothing else in that account. Once the submission server is deployed in Part 6, **nobody else is asked at all**: cadets and cadre sign in with Google to prove who they are, and reach their data through the server rather than through Drive. A verified badge changes who vouches for the app's identity; it does not widen what the app may reach.

## 3 Open the app

There is nothing to download, install or host. 9ThirtyOne is published at one address, and every detachment uses that same address.

```
https://9thirtyone.app/
```

Open it in any current browser and go on to Part 4. What makes the install *yours* is not a copy of the app, and not the Google registration either — every detachment shares that one. It is the **Drive folder**: it sits on your own Google account, holds every record, and nobody else can reach it.

> **The same address for everyone, cadets included**
>
> Cadre, cadets and administrators all open this one address. Nobody needs a copy of their own, and the join links and QR codes in Part 8 are built from it for you — so this is the last time you will need to type it.

> **Why you do not host it yourself**
>
> A detachment-hosted copy would mean registering a second origin with Google, keeping that copy patched, and repeating both at every update — for no gain, because **the app holds no data**. Every record belongs to the Drive folder on your account, so sharing an address with other detachments gives up nothing in privacy or control. They cannot see your folder and you cannot see theirs.

> **Updates arrive on their own**
>
> Because everyone loads the same published copy, fixes reach the detachment with nothing to install. If a device seems to be showing an old version, close every tab of the app and open the address again.

## 4 Run the setup wizard

> **If you are signed into more than one Google account, use a clean browser profile**
>
> Setup authenticates *twice* — once so the app can reach Drive, once to say who you are — and each time Google shows an account chooser that defaults to whichever account you signed in with first. Picking the wrong one at either prompt produces a half-configured install that fails confusingly later. A clean browser profile, or a private window signed into only the detachment account, removes the choice entirely. If your Drive address bar shows `/u/1/` or higher, you are not on your default account.

Open `https://9thirtyone.app/` in a browser. On a folder that has never been set up, the wizard starts automatically.

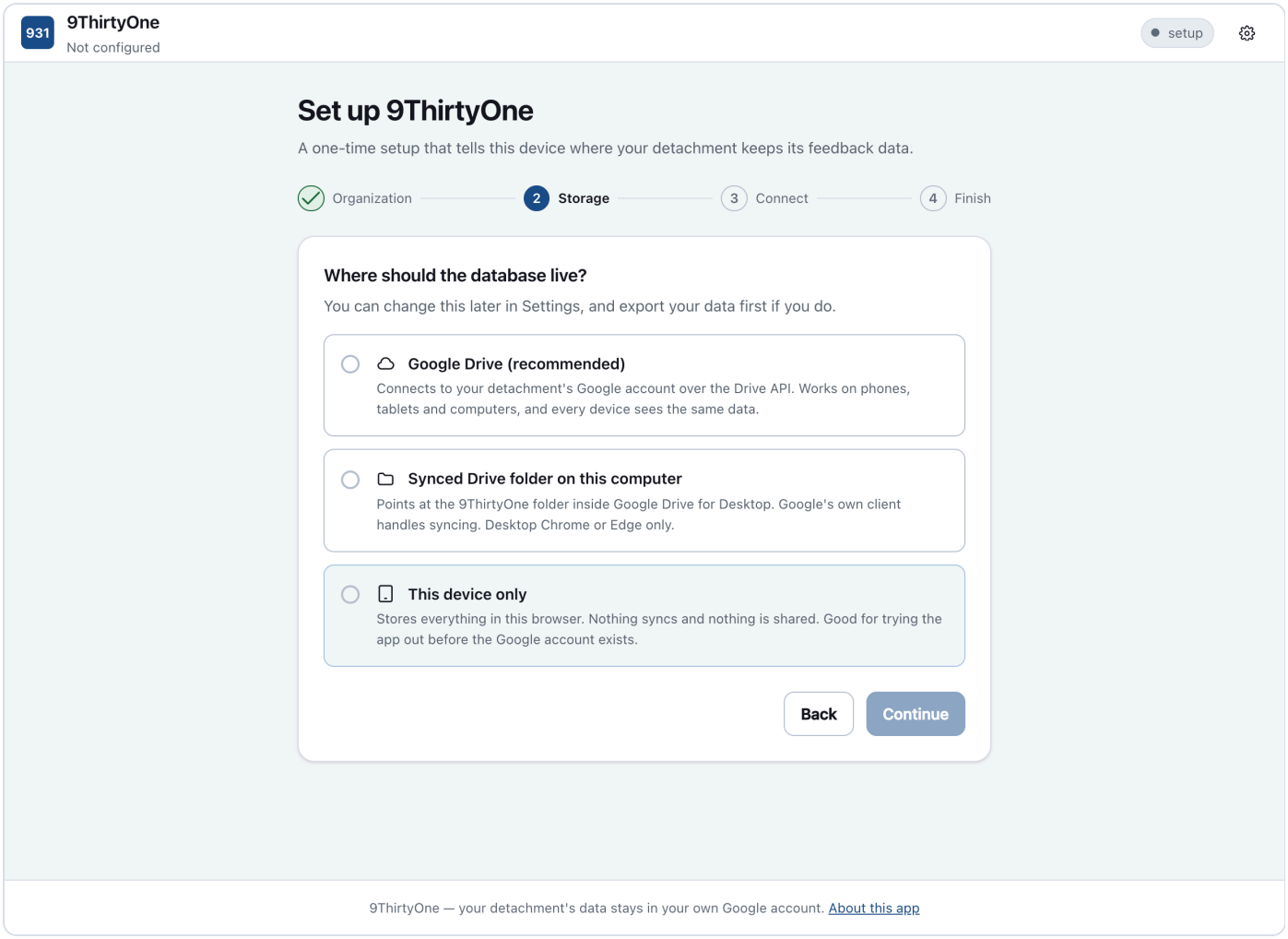
1. **Name the detachment.**
   This appears in the app header and on exported reports.

2. **Choose your storage option**
   from Part 1.

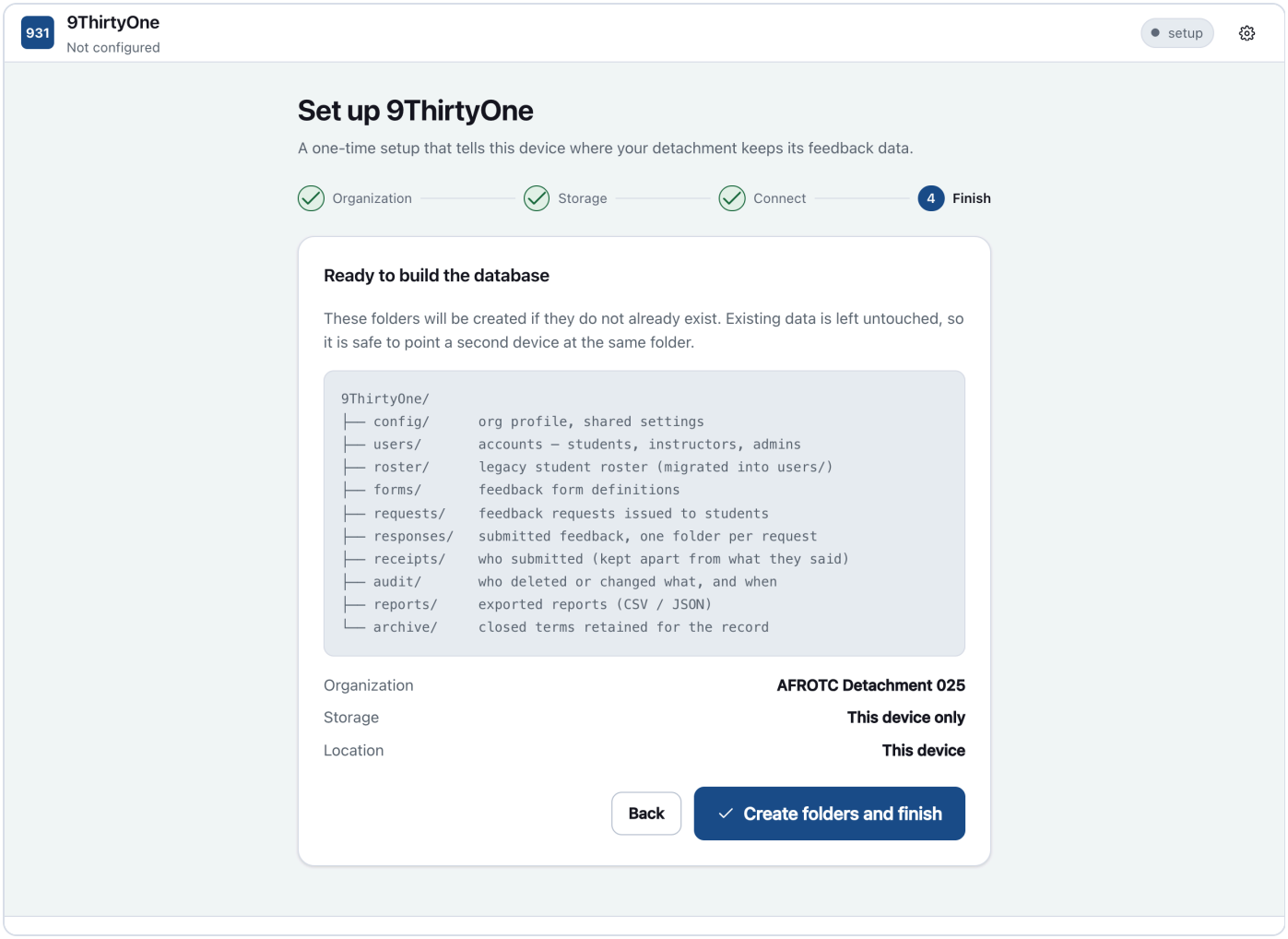
3. **Connect.**
   - **Google Drive:** click *Sign in and set up the folder* — there is nothing to paste first. A Google window opens; sign in as the detachment account and grant access. The app creates its `9ThirtyOne` folder and shows you its address — **copy that address now**, you need it in Part 6. There is no folder link to paste: see Part 2a for why.

     If this account already has a `9ThirtyOne` folder from a previous setup, the wizard lists it with the date it was made and offers to use it. **Take the offer.** Creating a second folder does not move anything into it — your records stay where they are and the app starts an empty detachment, which looks from the inside exactly like having lost them.
   - **Synced folder:** click *Select folder…* and choose the 9ThirtyOne folder inside your Google Drive for Desktop directory.
   - **This device only:** nothing to connect.

4. **Create the folders.**
   The wizard shows exactly what it will create and then builds it. Existing data is never touched, so it is safe to point a second device at the same folder and run the wizard again.




Step 2 of the wizard. Options unavailable on the current device are greyed out with the reason.




Step 4 shows the folder structure before creating it. Each record is a plain JSON file you can read in Drive.

## 5 Claim the folder

9ThirtyOne issues no passwords. Everyone signs in with the Google account your detachment already mails them at, and a roster inside your Drive folder decides who is allowed in and what they can do. Your first job is to put yourself on that roster.

A folder with an empty roster is claimed by **the first Google account to sign in**, which becomes both a database administrator and an instructor. After that the roster is closed: an account that is not on it is turned away, whoever it belongs to.

1. **Leave the folder shared with nobody.**
   You do not need to share it, and sharing it would not help anyone use the app — see the note below.
2. **From the home screen, open *Database Administration*.**
   It should say this detachment has no roster yet.
3. **Click Sign in with Google**
   and choose the account you want to administer the detachment with.
4. **Confirm you landed in the console**
   and that your email appears on the roster with the *Database admin* and *Instructor* roles.
5. **Add a second administrator.**
   The console will remind you while you are the only one. It is not a formality: a single admin who loses access leaves the roster repairable only by editing `users/users.json` in Drive by hand.
6. **Check the access setting.**
   Go to *Settings* → *Access*. It should read "Google sign-in only".

> **Sharing the folder does not give anybody access to the app**
>
> Worth getting right, because an earlier version of this guide said the opposite. The app asks Google only for the files *it created itself*, so it can open this folder for the account that made it and for nobody else. Share the folder with a colleague and the app still will not open it for them — what they get instead is raw access to the Drive folder, where every response sits in the open, including the anonymous ones.
>
> **So Drive sharing is not the access control. The roster is**, and from Part 6 it is enforced by the submission server rather than by Drive. Add people to the roster; leave the folder to yourself.
>
> Claiming is the one exception, and it closes on its own. A roster with nobody on it is taken by the first Google account to sign in — which is how a new detachment gets in without a shared password printed in a guide like this one. Only an account that can already reach the folder can do it, which at this point is yours alone; and once the server is deployed the door shuts for good, because an address that is not on the roster is refused outright.

**"Allow signing in with an email instead of Google"**

The one setting under *Settings* → *Access*. It exists for an installation with **no Google Client ID** — evaluating the app on a single device, before the Google setup in Part 2 has been done. Without it such an installation is a dead end, because Google cannot sign anybody in when there is no Client ID to sign them in against.

**It grants nothing on its own.** Whoever signs in that way is checked against the roster exactly like a Google sign-in: an address nobody has added is refused, and the roles that come back are the real ones. It adds a way to say who you are; it is not a way to skip being anybody.

A detachment on Google Drive has a Client ID by definition, so the sign-in screen will not offer it and the setting should stay off. Once your detachment has an administrator account, only an administrator can switch it on — turning it *off* is never restricted.

## 6 Deploy the submission server

> **This part is no longer optional**
>
> The app asks Google only for access to files it created itself, which means a cadet's phone has no way to reach your detachment's folder — it did not create it. The submission server is what files their feedback for them. **Without it, join links cannot set up anyone else's device.**
>
> Earlier versions could fall back to giving every cadet access to the whole folder. That was the arrangement where any cadet could open Drive and read every response, and it is gone.

Fifteen minutes, and it changes what everything else costs. A small script running inside your own Google account files everyone's feedback for them, so nobody but you needs access to the Drive folder.

> **Without this, everyone who uses the app can read every response**
>
> Google requires *Editor* permission to write a file, and Drive has no write-without-read. So without the server, every cadet who submits feedback can also open the folder and read every response in it, including anonymous ones — and can alter them, or delete the activity log. The anonymity in the app's own screens is real and worth nothing against somebody who opens Drive directly.

With it deployed, three things become true at once:

- **Nobody but you needs Drive access.** Cadets and cadre both reach their data through the server.
- **Nobody sees a Google permission screen asking for their Drive.** The app stops asking for it entirely, which also removes the "*Google hasn't verified this app*" warning your cadets would otherwise meet before they ever saw the product.
- **One submission per cadet becomes a rule rather than a convention**, because the server checks and writes under a lock. Two taps on a bad connection cannot both land.

### Deploy it

1. **Open script.google.com**

   signed in as the account that owns the Drive folder. This matters: the script files everything using that account's access.

2. **New project**

   , named `9ThirtyOne Server`. Delete what is there and paste in `tools/proxy/Code.gs`, downloaded from the project's public repository at `github.com/dmg-arts/9thirtyone` — open the file and use its *Raw* view, so you copy the script and not the page around it.

3 **Project Settings → Script Properties**

— the gear icon in the left sidebar. Add these two, then *Save script properties*. The names are case-sensitive.

| PROPERTY | VALUE |
|---|---|
| `FOLDER_ID` | Your folder's id. The wizard showed it when it created the folder, and it is in Drive's address bar after `/folders/`. |
| `CLIENT_ID` | `536562746421-uf9hu736m0bq0t41s0ck8h9fmu9ggkos.apps.googleusercontent.com` — the same for every detachment. Copy it from here. |

4 **Run `checkSetUp` once**

from the editor. It confirms the folder is reachable and, more importantly, makes Google ask for the Drive permission the script needs — setting the properties by hand never prompts for it, and without it the first cadet submission is where you would find out.

5 **Deploy → New deployment → Web app**

, set to *Execute as: Me* and *Who has access: Anyone*, and copy the address it gives you. It ends in `/exec`.

> **"Anyone" is the one to get right**
>
> Not *Anyone with a Google account* — the two sit next to each other in the list. Cadets reach this endpoint without a Google session, so the stricter option refuses every submission with an error the app can only report as *could not reach*.

6 **Paste it into Settings → Submission server**

and press *Save and test*. It checks the deployment is reachable, is the right script, and is configured, so you find out now rather than when a cadet cannot submit.

> **"Anyone" sounds wrong. It is not.**
>
> The address is public, and the script refuses everything that does not carry a valid Google sign-in for somebody on your roster — it checks that with Google on every single request. Access is decided by your roster, not by the address being hard to guess. Cadets are not part of any organisation account, so there is no narrower setting that would work.

**The first call after a quiet spell is slow**

Google lets an idle deployment go to sleep. The first request after some hours of no use has to wake it, which takes a few seconds and occasionally long enough to give up — usually the first sign-in of the day, or the first one after you deploy. Everything after it is immediate, because the script is then warm.

So a sign-in that appears to do nothing, or reports that the submission service did not answer in time, is worth **one more attempt** before you go looking for a fault. Wait for it rather than pressing again: each press starts its own sixty-second request instead of hurrying the one already running, so repeated pressing makes a slow sign-in look like a broken one.

This is a property of Apps Script, not a misconfiguration, and there is no setting that turns it off. It does not affect cadets filling in feedback during a drill period — by then the script has been woken by whoever signed in first.

**Full instructions live with the script**

`tools/proxy/README.md` — on GitHub, at `github.com/dmg-arts/9thirtyone` — covers each step with screenshots of the settings, what every error message means, and how to redeploy after a change — forgetting to redeploy is the most common reason an edit appears to do nothing.

## 7 Build the roster

Everyone signs in — cadets included. That is what makes "one submission per cadet" mean something rather than relying on an honour system.

| ROLE | CAN DO |
|---|---|
| **Cadet** | See feedback assigned to them and submit it once. |
| **Instructor** | Create feedback, read responses, run analysis. In *By instructor*, their own results. |
| **Cadre** | Everything an instructor can, plus the *Cadre Panel* — the same screen again, holding feedback instructors cannot see. Not a hidden tab: a separate folder they have no access to. In *By instructor*, cadre see the whole instructor pool, plus their own results as an instructor — but not another cadre member's. Only a commander sees the cadre. |
| **Commander** | Sees every space including its own, which nobody else can read. Their own space appears inside the Cadre Panel, marked. In *By instructor* a commander sees everyone, cadre included. **At most two at a time**, so a change of command overlaps. |
| **Database admin** | Add and remove people, maintain the database, delete feedback — always with a recorded reason. |

**Two panels, not one list**

The **Instructor Panel** holds the detachment's ordinary feedback. The **Cadre Panel** is the same screen — same tabs, same filters, same form creator — pointed at the restricted folders. Anyone holding cadre can move between them with a button in the top corner, and feedback created from a panel is filed into that panel's space automatically.

They are separate rather than one list with a lock icon for a practical reason: the detachment list can then be read on a shared screen without checking who is standing behind you.

When someone creates feedback they choose which space it belongs to. Detachment feedback is visible to every instructor; cadre-only feedback is not; commander-only feedback is visible to the commander alone. These are separate folders on Drive, enforced by the submission server, so "locked" means locked rather than hidden.

Cadets are asked to fill in feedback from every space — a commander's request is still meant to be answered — and never see anybody's responses, so being asked reveals nothing.

**Two commanders, deliberately**

Not one, so a change of command overlaps rather than cutting over: the outgoing and incoming commander both hold it during the handover. The space belongs to the detachment, so handing over means moving the designation — no records move, and nothing is lost.

> **Which email address?**
>
> The one your detachment actually mails the cadet at. At a regional detachment that is usually a Gmail address, even for cadets whose school address ends in `.edu` — many colleges run their mail through Google, and the cadet reads it in Gmail. If in doubt, use the address already in your distribution list. It is the address the cadet will click "Sign in with Google" as.

### One at a time

1. **Open Database Administration**
   and click *Add person*.
2. **Enter their name and their Google account email.**
3. **Tick the roles they need**
   and choose their AS level.
4. **Save.**
   There is nothing to hand them — they can sign in immediately.

### A whole class at once

1. **Prepare a CSV**
   with a `name` column and an `email` column. That much creates cadets, which is what a new intake needs. Optional columns: `roles`, `class`, `section` and `username` — the same six *Export CSV* writes, so an exported roster can be edited in a spreadsheet and imported back without losing anybody's role.
2. **Click Import roster CSV**
   and choose the file.
3. **Fix anything it could not add.**
   A duplicate address, a typo, an unknown role or an unknown AS level is listed with the reason, and that row is left out rather than guessed at. Correct the file and import again — rows already added are skipped rather than duplicated.

**Let an assistant do the formatting**

`docs/ROSTER-FORMAT.md` in the project on GitHub states the format precisely and includes a prompt written to be handed to an AI assistant along with `docs/roster-template.csv` and whatever list you already have — a spreadsheet, an email, a printout you have typed up. It comes back as a file you can import.

**One rule if you do that: never let it invent an email address.** The address is the identity — it is what a Google sign-in is matched against — so a plausible guess creates an account the real person can never sign in to, and another one for you to find and delete. The prompt tells it to leave those people out and list them separately instead.

### Telling them how to get in

Adding someone to the roster does not notify them. The one message this app can send is an optional notice when a feedback request opens, which is off unless an administrator switches it on — see `tools/proxy/README.md`. Once they are on the list, send them the join link from *Invite people* at the top of this same screen. **Copy link** for chat, **Email it** to open a pre-written message, or **Share** on a phone.

For a whole flight at once, **Show QR code** fills the screen with a scannable code. Put it on a projector at the start of a meeting and have cadets point their cameras at it — no app needed, the camera offers to open it. It is the fastest way to set up thirty phones, and nobody types anything.

### When someone changes email

Edit their account and change the address. Their submission history follows them: receipts are filed under an internal username such as `alvarez.mia`, which does not change when the email does.

### When someone leaves

Untick **Active** rather than deleting them. A deactivated account cannot sign in, but the record stays, so past feedback still makes sense.

**Deleting goes further, permanently.** Their feedback is kept, but their name is stripped out of it for good: every response they wrote becomes anonymous, and their submission receipts stop identifying them while still counting toward completion. This cannot be undone, and restoring a backup taken afterwards does not bring it back. It is the right choice when somebody asks to be removed; it is not a way to tidy the roster.

**There is nothing to reset**

If a cadet cannot get in, it is one of three things: the address on the roster is not the one they are signing in with, their account is marked inactive, or they do not have the role for the screen they opened. All three are visible in *Database Administration*. This app has no password to forget.

# 8 Invite everyone

Everything before this was for your device. Everyone else gets a link.

> **Nobody else runs the setup wizard**
> A join link carries the Client ID, the folder and the submission server, so tapping it sets a device up completely. Cadets sign in with Google and land on their feedback. Nothing to paste, nothing to explain, and no Client ID in the hands of a nineteen-year-old on a phone.

### Sending the link

Go to *Database Administration → Invite people*. Three ways out, all the same link:

- **Copy link** for whatever your detachment already uses to talk — group chat, email, the LMS.
- **Email it** opens a pre-written message that explains what to expect.
- **Share** on a phone hands it to the usual share sheet.

### A whole flight at once

**Show QR code** fills the screen with a scannable code. Put it on a projector at the start of a meeting and have cadets point their cameras at it — no app needed, the camera offers to open it. It is the fastest way to set up thirty phones, and nobody types anything.

> **The link is not a password**
> Everything in it is public or an address. The Client ID appears in the page source of every app that uses one, and the folder ID is not a key. Opening the link grants nothing on its own: the roster decides who may sign in, and an address that is not on it is turned away. Post it wherever is convenient.

### Adding it to a home screen

| DEVICE | HOW |
|---|---|
| iPhone / iPad | Open the link, tap **Share**, then *Add to Home Screen*. Safari and Chrome both work. |
| Android | Chrome offers *Install app* in its menu, or prompts on its own. |
| Mac / Windows | Chrome and Edge show an install icon in the address bar. |

> **Add the icon first, then sign in**
> A home-screen app gets its own private storage, separate from the browser it was added from. Open the join link, add the icon, and *then* sign in from the icon — doing it the other way round means signing in twice.

## 9 Verify and hand over

Work through this before letting cadets use the app for real.

- [ ] **The app opens** at `https://9thirtyone.app/`.
- [ ] **Settings → Access reads "Google sign-in only"** — the email sign-in is off.
- [ ] **At least two people hold the Database admin role**, so losing one account does not lock the detachment out.
- [ ] **The submission server answers** — Settings → Submission server → *Save and test* reports a version.
- [ ] **The Drive folder is shared with nobody but you.** With the server deployed, nobody else needs it.
- [ ] **The connection indicator in the header reads "ready"** on a device other than the one you set up on.
- [ ] **The Drive folder contains the app's folders** — open it in Drive and confirm you can see `config`, `users`, `forms` and the rest.
- [ ] **A test cadet can sign in with their own Google account** and see feedback assigned to them — on their own phone, not yours, reached by tapping a join link. If this is the first thing to touch the server in hours, give the first attempt time and try once more before treating it as a failure — see Part 6.
- [ ] **That cadet was never asked for access to their Google Drive.** If they were, the submission server is not configured on their device — re-send the join link.
- [ ] **A test submission appears** in the Instructor Panel under Responses & analysis.
- [ ] **You have exported a backup** from Instructor Panel → Database → Export backup.
- [ ] **You have taken one anonymised export** and looked at it, so you know what is in the backup copy you keep off-site.

### Moving trial data into Drive

If you evaluated using *This device only*:

1. On the trial device, go to *Instructor Panel → Database → Export backup* and save the JSON file.
2. Go to *Settings → Re-run setup* and connect to Google Drive as in Part 4.
3. Return to *Database → Import backup*, choose the file, and select *Merge*.

### Routine maintenance

| TASK | HOW OFTEN | WHERE |
|---|---|---|
| Export a backup | End of each term | Instructor Panel → Database |
| Review accounts, deactivate leavers | Start of each term | Database Administration |
| Rebuild indexes (only if counts look wrong) | Rarely | Database Administration |
| **Advance the academic year** — every cadet up one AS level, the graduating class deactivated. No undo, so export a backup first. If the year has already been advanced the card says so and asks again before proceeding. | **Once**, at the start of the year | Database Administration |

# Troubleshooting

| WHAT YOU SEE | WHAT TO DO |
|---|---|
| "Could not connect" when signing in to Google | Almost always the network rather than the configuration — there is nothing on your side to register any more. Check you are on `https://9thirtyone.app` and not a copy served from somewhere else, then try again. If it persists on a detachment network, a content filter is blocking `accounts.google.com`. |
| Sign-in takes several seconds, or seems to hang the first time | Expected, once. The submission server sleeps when nobody has used it, and the first sign-in of the day has to wake it. The screen shows a count of seconds while it does. Wait rather than pressing again — each press starts its own request instead of hurrying the one already running. See Part 6. |
| The sign-in popup does not open | Popups are blocked for the site. Allow them and try again. |
| Sign-in popup closes immediately, or `origin_mismatch` | The app is being served from an address Google does not recognise — a local copy, a fork on someone else's hosting, or an old bookmark. Use `https://9thirtyone.app`. If you are deliberately running your own copy you need your own Google registration and Client ID; see *Advanced* in the wizard's Connect step. |
| The home-screen icon opens the setup wizard again | Normal. A home-screen app has its own storage, separate from the browser. Run the wizard once inside the icon app and it will not ask again. |
| Connects, then Drive requests fail with 403 | Sign out of Google in *Settings*, then sign in again and grant access when asked. A 403 after a successful sign-in means the Drive permission was declined, or was later revoked for this account. |
| "The folder or file was not found" | Almost always a folder created by hand rather than by the app. The app can only reach files it made itself, so a folder you made in Drive is invisible to it however correct the address looks. If you have never run the wizard on this device, run it — it will offer any folder it made for this account before creating a new one. If you had a working install and it has lost its place, use *Settings* → *Storage* → *Change folder* instead: that reconnects to a folder you already have, where re-running setup risks starting an empty one beside it. |
| Header shows "offline" with a pending count | Normal. Work is saved on the device and sends automatically. Click the pending badge to retry now. |
| A cadet cannot see a form | Check its status is *Open*, the open date has passed, the due date has not, and the AS level matches theirs. If it targets named cadets, check they are on the list. |

| | |
|---|---|
| "Results withheld" instead of analysis | Working as designed. Anonymous results stay hidden until three people have responded, because a single response can be traced to its author by elimination. |
| "Not on this detachment's roster" | The address they signed in with is not the one on the roster. Ask what Google account they used and compare it with Database Administration — a school address and the Gmail account behind it are two different strings. |
| Nobody can sign in as an administrator | Open `users/users.json` in Drive, add `"admin"` to the `roles` array of an account you control, and save. This is the recovery path; it is why at least two people should hold the admin role. |
| Response counts look wrong | Database Administration → *Rebuild indexes*. Counts are a cache and can lag; this rebuilds them from the underlying records. |

## What this version cannot do yet

Installed and run end to end against real Google Drive. These are its known limits, stated plainly so you can plan around them.

| LIMIT | WHAT IT MEANS FOR YOU |
|---|---|
| **Sign-in depends on one shared registration** | Every detachment now signs in through a single verified Google application rather than one of its own. That is what removed the Cloud setup, the test-user list and the hundred-account cap — and it means a problem with that registration would affect every detachment at once, where a per-detachment one failed alone. |
| **Maintenance runs from the owner's device** | Backup, restore, wipe and reindex act on the whole folder, and the submission server deliberately offers no way to do any of them remotely — an address that could empty a detachment's records on request is not one worth having. Sign in on the account that owns the folder. |
| **Written feedback can still identify people** | No amount of removing names changes what somebody wrote. A cadet who says they are the only AS400 in their flight has identified themselves. This is true of every feedback system and is worth saying to cadets rather than promising more than can be delivered. |
| **The safety screen matches words, not meaning** | It cannot tell "we discussed hazing prevention" from "I was hazed", and a clear screen is not proof nothing was reported. It is a prompt to read something now, never a finding. |

**Limits that used to be here and no longer are**

Earlier versions of this guide warned that everyone who submitted feedback needed Editor access to the Drive folder and could therefore read every response; that every device had to be set up by hand with a Client ID pasted in; and that every user met an alarming Google permission screen. The submission server and join links removed all three. If you are reading an older copy of this guide, replace it.

# Reference

### What gets created in your folder

```
9ThirtyOne/
├── config/      detachment profile and shared settings
├── users/       the roster — who may sign in, and as what
├── forms/       feedback form definitions
├── requests/    feedback issued to cadets, each with an ID
├── responses/   submitted feedback, one folder per request
├── receipts/    who submitted (kept apart from what they said)
├── audit/       every deletion and roster change, with who did it
├── roster/      legacy roster, migrated into users/ on setup
├── reports/     exported reports
└── archive/     closed terms retained for the record
```

Two more appear later: `cadre/` and `commander/`, holding feedback those roles alone can read. They are created the first time something is filed into them, so a folder you have just set up will not have them yet — their absence is not a fault.

### Things worth understanding before you field this

#### Anonymity is real, and it has a limit

On an anonymous form the cadet's sign-in is used to verify them and to write a *receipt*, then discarded. The response record carries no name, and receipts live in a different folder. This is what lets the app say "two cadets still owe feedback" about responses nobody can attribute. The limit: with very few responses, a single answer can be identified by elimination, so anonymous results stay hidden until three people have responded.

#### The cadre and commander spaces are separate folders

Not hidden screens. `cadre/` and `commander/` are real folders that the submission server will not open for somebody without the role, which is why "locked" here means locked. Feedback cannot be moved between spaces once it exists, because moving it would carry its responses somewhere they were never meant to be readable.

#### Sign-in is not encryption

Anyone with access to the Drive folder can read every record directly, regardless of what the app shows them. The folder is shared with nobody, so that means whoever can sign in to the Det Google Account. Keep that account to as few people as possible — it is the access control.

The app stores no credentials at all — `users/users.json` holds emails and roles, nothing secret. The identity check is Google's. The app validates the sign-in token in the browser, which catches an expired or misdirected one but is not a defence against someone who controls the browser. The check that holds is the submission server's: it verifies the token against Google and checks the roster before it writes or returns anything.

#### The safety screen is a prompt, not a finding

Written answers are automatically checked for language about hazing, harassment, discrimination, violence, self-harm, substances and integrity. A match means read that response now — it cannot tell "we discussed hazing prevention" from "I was hazed". A clear screen is not proof that nothing was reported. Follow your detachment's own reporting procedures.

#### Safety alerts override the anonymity threshold

If a flagged answer appears on a form whose results are otherwise withheld, the app tells you it exists and lets you open it behind a warning that doing so may identify the author. A disclosure cannot wait for a third response to arrive.

### Getting help

Diagnostics are available at *Settings → Download diagnostics*. The file describes how this installation is behaving: which version of the app and of your submission server are running, whether the folder layout is intact, how long the server has been taking to answer, and what has failed and how often.

It carries **no roster, no feedback and no free text of any kind** — not your detachment's name, not your folder address, not a single cadet's name or answer, and no timestamp more precise than a month. The file says so at the top, in a list of what was left out. Open it and read it before you send it; it is short, and it is meant to be read.

Send it to `david.mark.gaspar@gmail.com`, with what you were doing when the problem appeared. Questions from a cadet about their own feedback go to your detachment's administrator instead — they hold the records, and we do not.